```
!pip install -q kaggle

from google.colab import files
uploaded = files.upload()

!mkdir -p ~/.kaggle
!cp kaggle.json ~/.kaggle/
!chmod 600 ~/.kaggle/kaggle.json

!kaggle datasets download -d jimschacko/airlines-dataset-to-predict-a-delay
!unzip airlines-dataset-to-predict-a-delay.zip
```

```
Choose Files No file chosen
cp: cannot stat 'kaggle.json': No such file or directory
chmod: cannot access '/root/.kaggle/kaggle.json': No such file or directory
Dataset URL: https://www.kaggle.com/datasets/jimschacko/airlines-dataset-to-predict-a-delay
License(s): CC0-1.0
Downloading airlines-dataset-to-predict-a-delay.zip to /content
100% 6.15M/6.15M [00:00<00:00, 105MB/s]

Archive:  airlines-dataset-to-predict-a-delay.zip
  inflating: Airlines.csv
```

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix

df = pd.read_csv('Airlines.csv')
display(df.head())
df = df.dropna()

features = ['Airline', 'Flight', 'AirportFrom', 'AirportTo', 'DayOfWeek', 'Time', 'Length']
target = 'Delay'

X = df[features].copy()
y = df[target].copy()
```

	id	Airline	Flight	AirportFrom	AirportTo	DayOfWeek	Time	Length	Delay
0	1	CO	269	SFO	IAH	3	15	205	1
1	2	US	1558	PHX	CLT	3	15	222	1
2	3	AA	2400	LAX	DFW	3	20	165	1
3	4	AA	2466	SFO	DFW	3	20	195	1
4	5	AS	108	ANC	SEA	3	30	202	0

```
encoders = {}
categorical_columns = ['Airline', 'AirportFrom', 'AirportTo']

for col in categorical_columns:
    le = LabelEncoder()
    X[col] = le.fit_transform(X[col])
    encoders[col] = le

print("Categorical variables encoded successfully.")
display(X.head())
```

```
Categorical variables encoded successfully.
```

	Airline	Flight	AirportFrom	AirportTo	DayOfWeek	Time	Length
0	4	269	253	135	3	15	205
1	14	1558	217	60	3	15	222
2	1	2400	154	80	3	20	165
3	1	2466	253	80	3	20	195
4	2	108	14	252	3	30	202

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

print(f"Training data shape: {X_train.shape}")
print(f"Testing data shape: {X_test.shape}")

rf_model = RandomForestClassifier(n_estimators=100, random_state=42, n_jobs=-1)

print("Training the model... this might take a minute.")
rf_model.fit(X_train, y_train)
print("Model training complete!")
```

```
Training data shape: (431506, 7)
Testing data shape: (107877, 7)
Training the model... this might take a minute.
Model training complete!
```

```
# Make predictions on the test set
y_pred = rf_model.predict(X_test)

# Calculate Accuracy
accuracy = accuracy_score(y_test, y_pred)
print(f"Model Accuracy: {accuracy * 100:.2f}%\n")

# Print Detailed Classification Report (Precision, Recall, F1-Score)
print("Classification Report:")
print(classification_report(y_test, y_pred))

# Plot the Confusion Matrix
cm = confusion_matrix(y_test, y_pred)
plt.figure(figsize=(6,4))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=['On Time', 'Delayed'],
            yticklabels=['On Time', 'Delayed'])
plt.xlabel('Predicted Status')
plt.ylabel('Actual Status')
plt.title('Confusion Matrix')
plt.show()

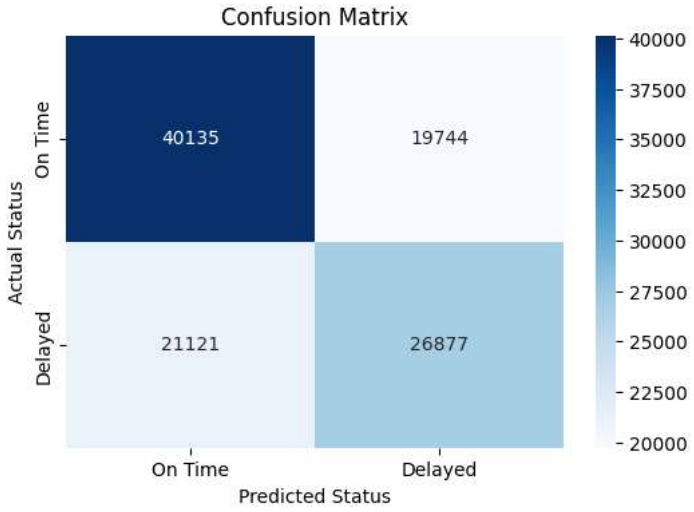
# Bonus: Feature Importance Plot
importances = rf_model.feature_importances_
sns.barplot(x=importances, y=features)
plt.title('Feature Importance in Predicting Flight Delays')
plt.show()
```

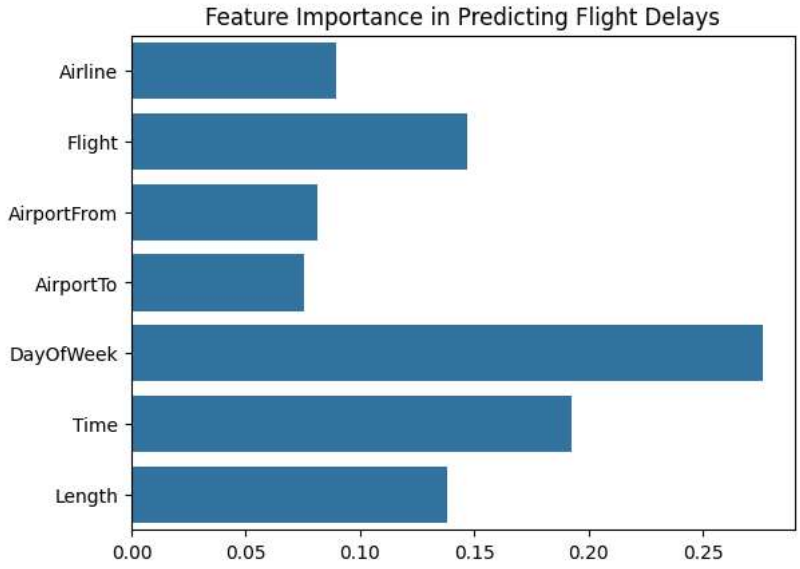
```
Model Accuracy: 62.12%

Classification Report:
              precision    recall  f1-score   support

           0       0.66      0.67      0.66     59879
           1       0.58      0.56      0.57     47998

    accuracy                           0.62    107877
   macro avg       0.62      0.62      0.62    107877
weighted avg       0.62      0.62      0.62    107877
```






```
def predict_flight_status(airline, flight_num, origin, dest, day_of_week, time, distance):
    """
    Takes user input, encodes it, and returns a delay prediction.
    """
    # 1. Structure the input as a DataFrame
    input_data = pd.DataFrame({
        'Airline': [airline],
        'Flight': [flight_num],
        'AirportFrom': [origin],
        'AirportTo': [dest],
        'DayOfWeek': [day_of_week],
        'Time': [time],
        'Length': [distance]
    })

    # 2. Encode the categorical inputs using our saved encoders
    try:
        for col in categorical_columns:
            input_data[col] = encoders[col].transform(input_data[col])
    except ValueError as e:
```

```
        return f"Error: Unrecognized input for Airline or Airport. Ensure it matches the dataset."

    # 3. Make the prediction
    prediction = rf_model.predict(input_data)

    # 4. Format the output
    if prediction[0] == 1:
        return "Predicted Status: Delayed"
    else:
        return "Predicted Status: On Time"

# --- Test the Sample Output ---
# Note: Use Airline and Airport codes that actually exist in your chosen dataset
print("\n--- User Input Prediction ---")

test_airline = 'DL'         # Delta Airlines
test_flight_num = 1234
test_origin = 'ATL'         # Atlanta
test_dest = 'JFK'           # New York
test_day = 3                # Wednesday
test_time = 1110            # 18:30 in minutes from midnight
test_distance = 1200

result = predict_flight_status(
    test_airline, test_flight_num, test_origin, test_dest, test_day, test_time, test_distance
)

print(f"Input: Airline={test_airline}, Route={test_origin}->{test_dest}, Time={test_time}")
print(f"Output: {result}")
```

```
--- User Input Prediction ---
Input: Airline=DL, Route=ATL->JFK, Time=1110
Output: Predicted Status: On Time
```

```
import pickle

# Bundle the model and the encoders together into a dictionary
saved_data = {
    'model': rf_model,
    'encoders': encoders
}

# Save it as a .pkl (pickle) file
with open('flight_model.pkl', 'wb') as file:
    pickle.dump(saved_data, file)

print("Model saved successfully!")
```

```
Model saved successfully!
```

```
# 1. Install Streamlit and localtunnel
!pip install -q streamlit
!npm install localtunnel
```

```
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 9.1/9.1 MB 9.5 MB/s eta 0:00:00
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 11.3/11.3 MB 12.2 MB/s eta 0:00:00
⠙⠹ ⠙⠹ ⠙⠹ ⠙⠹
added 22 packages in 3s
⠸
⠸3 packages are looking for funding
⠸  run `npm fund` for details
⠸npm notice
npm notice New major version of npm available! 10.8.2 -> 11.13.0
npm notice Changelog: https://github.com/npm/cli/releases/tag/v11.13.0
npm notice To update run: npm install -g npm@11.13.0
npm notice
⠸
```

```
# 1. Install Gradio
!pip install -q gradio pandas scikit-learn

import gradio as gr
import pandas as pd
import pickle

# 2. Load the trained model and encoders
```

```
with open('flight_model.pkl', 'rb') as file:
    data = pickle.load(file)

model = data['model']
encoders = data['encoders']

# 3. Define the prediction function
def predict_delay(airline, origin, dest, day, time, distance):
    # Structure the user input
    input_df = pd.DataFrame({
        'Airline': [airline],
        'Flight': [9999], # Dummy flight number for the model
        'AirportFrom': [origin],
        'AirportTo': [dest],
        'DayOfWeek': [day],
        'Time': [time],
        'Length': [distance]
    })

    # Encode the categorical variables
    try:
        input_df['Airline'] = encoders['Airline'].transform(input_df['Airline'])
        input_df['AirportFrom'] = encoders['AirportFrom'].transform(input_df['AirportFrom'])
        input_df['AirportTo'] = encoders['AirportTo'].transform(input_df['AirportTo'])
    except ValueError:
        return "Error: Invalid Airport or Airline selected."

    # Make prediction
    prediction = model.predict(input_df)[0]
    probability = model.predict_proba(input_df)[0].max() * 100

    # Format output
    if prediction == 1:
        return f"Predicted Status: Delayed ({probability:.1f}% Confidence)"
    else:
        return f"Predicted Status: On Time ({probability:.1f}% Confidence)"

# 4. Build the Web Interface
interface = gr.Interface(
    fn=predict_delay,
    inputs=[
        gr.Dropdown(choices=list(encoders['Airline'].classes_), label="Airline"),
        gr.Dropdown(choices=list(encoders['AirportFrom'].classes_), label="Origin Airport"),
        gr.Dropdown(choices=list(encoders['AirportTo'].classes_), label="Destination Airport"),
        gr.Slider(minimum=1, maximum=7, step=1, label="Day of Week (1=Mon, 7=Sun)"),
        gr.Number(value=1110, label="Departure Time (Minutes from midnight, e.g., 18:30 = 1110)"),
        gr.Number(value=1200, label="Flight Distance (km)")
    ],
    outputs=gr.Textbox(label="Prediction Result"), # <--- Fixed line
    title=" Flight Delay Predictor",
    description="Select your flight details below to predict if your flight will be delayed.",
    theme=gr.themes.Soft()
)

# 5. Launch the app with a public link!
interface.launch(share=True, debug=True)
```

```
Colab notebook detected. This cell will run indefinitely so that you can see errors and logs. To turn off, set debug=False in la
* Running on public URL: https://18346db9527c1f0d5c.gradio.live

This share link expires in 1 week. For free permanent hosting and GPU upgrades, run `gradio deploy` from the terminal in the wor
```

